

Aspan Finance Audit Report

Table of Contents

1. Executive Summary	4
About Aspan	4
Audit Summary	4
Overall Security Posture	4
Launch Recommendations	5
Audit Scope	5
2. Assumptions and Considerations	6
Project & Protocol Assumptions	6
Centralization Risks	6
Trust Assumptions	6
3. Severity Definitions	7
Impact	7
Likelihood	7
Severity Classification Matrix	7
4. Findings	9
H01: SApUSD vault is vulnerable to first depositor attack	9
H02: Stale lista withdrawal index causes fund loss due to swap-and-pop reordering	11
H03: SApUSD.deposit() not blocked during multi-asset mode, allowing new depositors to steal xBNB from existing stakers	14
M01: Inconsistent Fee Tier check timing across PoolFacet functions can create wrong incentives	16
M02: Malformed PancakeV3 path allows an attacker to execute arbitrary external call from the Router	18
M03: 1 wei of xBNB present in the SApUSD will permanently DoS redeem() and withdraw() for all users	20
M04: Missing slippage check in redeemAndRequestUnstake	22
M05: Missing yield harvest before collateral increase in mintApUSD/mintXBNB inflates yield for sApUSD stakers	24
M06: getLSTExchangeRate ignores useIntrinsicValue flag, breaking admin emergency recovery for redemptions	26

M07: MEV protection bypass via direct SApUSD vault access	28
M08: Unbounded array copy in _requestListaWithdraw causes progressive DoS of native unstake path	30
L01: Absence of oracle fallback mechanism creates single point of failure for BNB price dependency	32
L02: Single global maxPriceAge exposes the protocol to stale prices, leading to DoS or value extraction	34
L03: apUSD minting and xBNB redeeming lacks post-state CR enabling forced conversion of the stability pool	36
L04: Deferred yield minting in _harvestYield still enables first-depositor capture of accumulated yield	38
L05: setLSTExchangeRateProvider does not reinitialize yield tracking, causing phantom or frozen yield	40
L06: cleanXbnb() selector not registered in diamond proxy, protocol recovery function unreachable	42
L07: mintXBNB does not enforce bootstrap, allowing first minter to capture existing equity at hardcoded 1 bnb price	44
L08: _getXBNBPriceBNB does not subtract pendingYieldUSD from xBNB backing, allowing xBNB redeemers to extract yield owed to sApUSD stakers	46
L09: Excess msg.value silently accepted and stuck in router	48

5. Enhancement Opportunities

50

E01: AspanRouter does not cover all four PancakeSwap V3 Fee Tiers	50
E02: Yield calculation uses raw collateralAmount without decimal normalization, causing incorrect yield for non-18-decimal LSTs	52
E03: Router overview claims PancakeSwap V2 support but only V3 is implemented	54
E04: Public deposit(uint256) described in spec but not implemented in WrappedClisBNB	55
E05: Documentation mismatch, minHarvestInterval default is 1 hour in code, not 0 as documented	57
E06: MEV protection uses block-based API but docs reference non-existent time-based API	58

About Us

60

About Adevar Labs	60
Audit Methodology	60
Confidentiality Notice	61
Legal Disclaimer	61

1. Executive Summary

About Aspan

Aspan is a decentralized stablecoin protocol built on BSC that allows users to mint apUSD (stablecoin) and xBNB (leveraged BNB exposure token) using liquid staking tokens as collateral. The protocol features dynamic fee tiers based on collateral ratio, yield distribution to stability pool stakers, and emergency stability modes to maintain protocol health during market stress.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	3	3	0
Medium	8	8	0
Low	9	8	1

Enhancement opportunities: 6

Overall Security Posture

The following sections describe the security posture before and after the fix review. The fix review is the stage of the audit where Adevar Labs checked the fixes implemented by the Aspan development team for the issues found during the audit.

Aspan built an ambitious DeFi protocol combining a diamond proxy architecture, dynamic fee tiers, and multi-modal stability mechanisms. The codebase has solid test coverage and reflects disciplined development practices. The complexity of the system introduced several vulnerabilities, all of which were resolved during this engagement.

After Fix Review Summary

The fix review confirmed successful resolution of all 3 High severity findings and 8 out of 8 Medium severity issues. The team addressed 8 out of 9 Low severity findings with 1 acknowledged. The Aspan team's implementation of the recommended fixes and their responsiveness to feedback demonstrates their commitment to code quality and security best practices. Notably, the team decided to go for a novel design for the SapUSD multi-asset vault, implementing a virtual offset for a multi-asset redeem function. Given the novelty of this approach, we strongly recommend comprehensive testing before deployment to production.

Before Fix Review Summary

The audit identified 3 High severity issues including SApUSD vault vulnerability to first depositor attacks, forced stability pool conversion through CR manipulation, and fund loss due to stale Lista withdrawal indices. 8 Medium severity issues were found related to oracle dependencies, fee tier inconsistencies, router vulnerabilities, and MEV protection bypasses. 9 Low severity issues involved

missing function registrations, yield calculation errors, and documentation mismatches. Additionally, 6 enhancement opportunities were identified to improve protocol robustness, documentation and user experience.

Launch Recommendations

After Fix Review Summary

All mandatory recommendations, and almost all strong recommendation have been successfully addressed. The team has implemented fixes for all High and Medium severity findings, demonstrating strong commitment to security. With 8 out of 9 Low severity issues addressed and comprehensive enhancement implementations.

Before Fix Review Summary

I. Mandatory before launch:

- Fix all High severity issues (SApUSD first depositor attack, stability pool forced conversion, Lista withdrawal fund loss)
- Fix all Medium severity issues including oracle fallback mechanisms, fee tier timing consistency, router path validation, and MEV protection bypass
- Register missing `cleanXbnb()` function selector in diamond proxy
- Implement proper yield harvest timing in mint functions
- Add slippage protection to `redeemAndRequestUnstake` function

II. Strongly recommended:

- Implement oracle fallback mechanisms for BNB price dependencies
- Fix yield calculation decimal normalization for future LST support
- Resolve documentation mismatches between code and specifications
- Add missing PancakeSwap V3 fee tier support (0.01% tier)
- Implement proper bootstrap enforcement for xBNB minting
- Add additional test cases for each reported issue

Audit Scope

- **Repository:** <https://github.com/aspan-finance/aspan-sc>
- **Before-Fix Review Commit Hash:** `edbbecc9583b48dbfc56d36d250c1a18f70b23f8`
- **After-Fix Review Commit Hash:** `5f7ae6653f162a032f18772b5f6bfa87e84d86f4`
- **Files/Modules in Scope:**
 - `src/**/*.sol`

2. Assumptions and Considerations

Project & Protocol Assumptions

- All current LSTs operations are 18-decimal tokens
- The protocol will be deployed with correctly configured Chainlink price feeds, LST registrations, fee tiers, and oracle bounds before being unpaused for public use.
- The WrappedClisBNB `YIELD_UPDATER_ROLE` holder will accurately compute off-chain yields and update the virtual exchange rate within the enforced safety constraints.
- The collateral ratio will be monitored and Stability Mode 2 triggered when CR falls below 130%.
- SApUSD stakers accept forced conversion risk to xBNB during Stability Mode 2 events.
- Appropriate `_minOut` slippage parameters will be provided as passing zero disables slippage protection.

Centralization Risks

- The Diamond Owner can:
 - Upgrade all protocol logic instantly via `diamondCut()` with no timelock, governance, or multi-sig enforced on-chain.
 - Change critical protocol addresses (apUSD, xBNB, treasury, stability pool, BNB price feed) without restriction.
 - Add or remove accepted LST tokens and change their exchange rate providers, price feeds, or manual prices.
 - Set fee tiers with fees up to 20%.
 - Pause and unpause the entire protocol, blocking all user deposits, withdrawals, minting, and redemptions.
- The Diamond Owner can transfer ownership to any address in a single step with no confirmation mechanism.
- The ApUSD and XBNB `DEFAULT_ADMIN_ROLE` holders can grant `MINTER_ROLE` to arbitrary addresses, enabling unlimited token minting.
- There is no emergency withdrawal mechanism for users when the protocol is paused.

Trust Assumptions

- Chainlink will provide accurate and timely BNB/USD price data; if the feed is unavailable or stale, all protocol operations are blocked.
- Lista DAO StakeManager and Astherus Minter will return accurate slisBNB exchange rates
- Lista DAO StakeManager will honor BNB staking deposits and withdrawal requests after the unbonding period.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

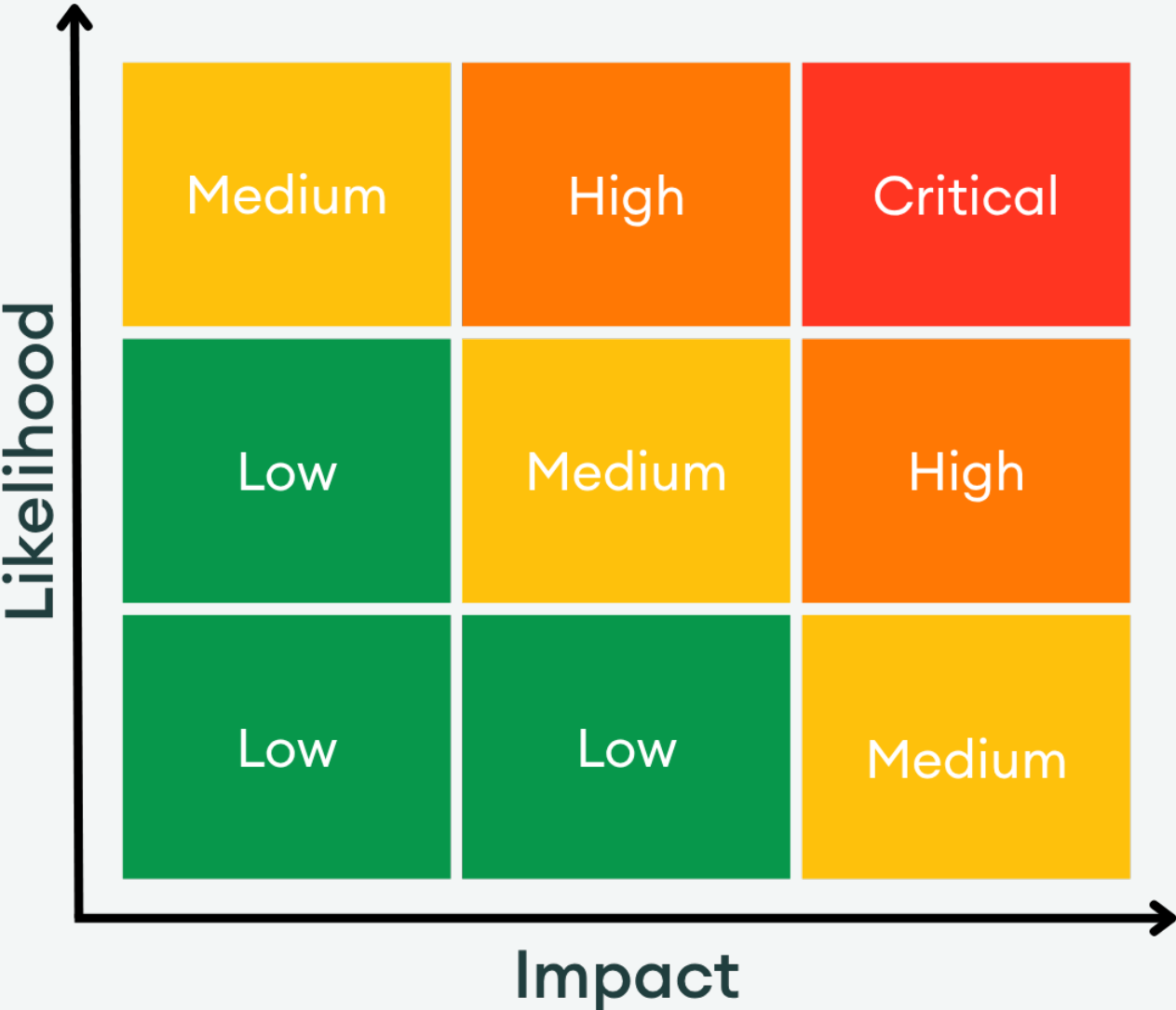
- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

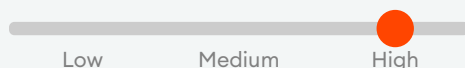
4. Findings

H01: SApUSD vault is vulnerable to first depositor attack

Status:

Resolved

Impact:



Likelihood:



Severity:

High

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/tokens/SApUSD.sol>

Description:

The first depositor (or inflation) attack is a well-known vulnerability of ERC-4626 vaults (see Open Zeppelin [article](#)).

What makes this attack possible is the ability for anyone to increase the asset's balance of the vault without increasing the shares balance, which has the effect of inflating the exchange rate for the next depositor.

The SApUSD contract inherits a lot of its logic from ERC-4626, but failed to implement the latest defenses against this attack vector, which are usually:

- dead shares
- virtual offset
- tracking token balance using a state variable rather than `ERC20::balanceOf` (see this great [article](#) from Euler)

The deposit function does check if (`shares == 0`) `revert Errors.ZeroAmount()`, but this guard only prevents the most extreme outcome of the attack (victim receiving exactly 0 shares).

It does not prevent the attacker from forcing the victim into a 1 share outcome while absorbing a large rounding remainder.

The fact that the BSC mempool is public allows an attacker to listen to incoming transactions and front-run a legitimate depositor in order to steal its deposit.

Proof of Concept:

The attacker exploits two properties in sequence: the 1:1 first-deposit branch and unlimited share price inflation via direct transfer.

1. Setup: Attacker deposits 1 wei of apUSD. Receives 1 share. **[State: totalAssets = 1, totalSupply = 1]**
2. Inflation: Attacker directly transfers `D` apUSD to the vault (bypassing deposit). **[State: totalAssets = D + 1, totalSupply = 1]**
3. Victim deposits `V` apUSD where $D < V < 2 \cdot (D+1)$, e.g., `D = 100 apUSD`, `V = 199 apUSD`:
 - `shares = 199 * 1 / 101 = 1` (integer division)
 - Check: `shares == 1 > 0`, transaction succeeds, victim receives 1 share
4. Redemption **[State: totalAssets = 300, totalSupply = 2]**, each share = 150 apUSD
 - Attacker redeems: receives 150 apUSD. **Profit: 150 - 1 - 100 = 49 apUSD**
 - Victim: deposited 199 apUSD, redeems for 150 apUSD. **Loss: 49 apUSD**

Recommendation:

The internal accounting introduced for M03 (tracking `accountedApUSD` and `accountedXBNB` via state variables instead of relying on `balanceOf`) also mitigates this attack, as direct token transfers to the vault would no longer inflate the exchange rate used in share conversion.

Additionally, implement a virtual asset/share offset in the conversion functions (`_convertToShares` and `_convertToAssets`) to protect against stealth donation attacks (i.e. rounding-based donation attacks that can still manipulate the exchange rate even with internal accounting).

See Open Zeppelin [ERC-4626 implementation](#) of the virtual offset.

The virtual offset ensures the exchange rate cannot be manipulated to extreme values even when the vault has very low share supply, making the first-depositor inflation vector economically impractical.

Fix Review Notes:

Fix 1: The implementation of internal accounting does solve the first depositor attack. The optional recommendation of virtual offset is not entirely implemented, as redeem only applies the virtual offset on the numerator, but not on the denominator. Also, after internal discussion, we (Adevar Labs) think that the virtual offset should be entirely removed, as this design might not be adapted to multi-assets vaults and is not well known yet, which could introduce side effects we're not yet aware of.

Fix 2: The team decided to still proceed with virtual offsets, now applying them symmetrically to both sides of the computation. We recommend thorough static and fuzz testing of the redeem function, particularly for scenarios involving repetitive deposit and redeem operations.

H02: Stale lista withdrawal index causes fund loss due to swap-and-pop reordering

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

High

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/periphery/AspanRouter.sol#L584-L597>

> _AspanRouter.sol

SOLIDITY

```
582:
583:    /// @notice Internal function to request withdrawal from Lista
584:    function _requestListaWithdraw(uint256 slisBNBAmount) internal returns (uint256 requestIndex) {
585:        // Approve Lista to spend slisBNB
586:        IERC20(slisBNB).approve(listaStakeManager, slisBNBAmount);
587:
588:        // Get current number of requests to determine the new index
589:        IListaStakeManager.WithdrawalRequest[] memory existingRequests =
590:            IListaStakeManager(listaStakeManager).getUserWithdrawalRequests(address(this));
591:        uint256 listaIdx = existingRequests.length;
592:
593:        // Request withdraw from Lista
594:        IListaStakeManager(listaStakeManager).requestWithdraw(slisBNBAmount);
595:
596:        // Store the request
597:        requestIndex = withdrawalRequests.length;
598:        withdrawalRequests.push(UserWithdrawalRequest({
599:            user: msg.sender,
```

Description:

The Router stores Lista's `userWithdrawalRequests` array index at request creation time and uses it verbatim when claiming:

SOLIDITY

```
// _requestListaWithdraw() - snapshots index once
uint256 listaIdx = existingRequests.length;
withdrawalRequests.push(UserWithdrawalRequest({ listaIdx: listaIdx, ... }));

// claimUnstake() - uses stale index
IListaStakeManager(listaStakeManager).claimWithdraw(request.listaIdx);
```

However, Lista's `_claimWithdrawFor()` uses **swap-and-pop** deletion:

SOLIDITY

... continued

SOLIDITY

```
// ListaStakeManager._claimWithdrawFor()
userRequests[_idx] = userRequests[userRequests.length - 1];
userRequests.pop();
```

Every successful claim moves the last element into the claimed slot, invalidating all previously stored indices.

Any user whose `claimUnstake` is not the last remaining request will have their BNB permanently stuck in Lista's StakeManager. With 2 requests, the second user's claim reverts (index out of bounds). With 3 or more requests, a stale index can silently point to a different user's request, enabling one user to claim another's withdrawal. There is no alternative claim path or admin recovery mechanism.

Recommendation:

Replace the mutable array index with Lista's stable `uuid` and resolve the current index at claim time:

DIFF

```
struct UserWithdrawalRequest {
    address user;
-   uint256 listaIdx;
+   uint256 listaUuid;    // stable identifier, immune to reordering
    // ...
}

function claimUnstake(uint256 requestIndex) external nonReentrant returns (uint256 bnbAmount)
{
    UserWithdrawalRequest storage request = withdrawalRequests[requestIndex];

+   // Resolve current array index from uuid
+   IListaStakeManager.WithdrawalRequest[] memory listaRequests =
+       IListaStakeManager(listaStakeManager).getUserWithdrawalRequests(address(this));
+   uint256 currentIdx = type(uint256).max;
+   for (uint256 i = 0; i < listaRequests.length; i++) {
+       if (listaRequests[i].uuid == request.listaUuid) {
+           currentIdx = i;
+           break;
+       }
+   }
+   require(currentIdx != type(uint256).max, "Request not found");

    uint256 balanceBefore = address(this).balance;
-   IListaStakeManager(listaStakeManager).claimWithdraw(request.listaIdx);
+   IListaStakeManager(listaStakeManager).claimWithdraw(currentIdx);
    bnbAmount = address(this).balance - balanceBefore;
    // ...
}
```

The `uuid` is a monotonically increasing global counter in Lista's StakeManager that uniquely identifies each request regardless of its current array position.

Fix Review Notes:

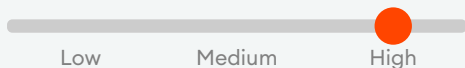
Fixed according to the recommendation, stale indices are no longer possible since the UUID is immutable and the mapping is re-synced after every claim.

H03: SApUSD.deposit() not blocked during multi-asset mode, allowing new depositors to steal xBNB from existing stakers

Status:

Resolved

Impact:



Likelihood:



Severity:

High

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/tokens/SApUSD.sol#L134>

>_ SApUSD.sol

SOLIDITY

```
132:      * @return shares Amount of shares minted
133:      */
134:      function deposit(
135:          uint256 assets,
136:          address receiver
```

Description:

After Stability Mode 2, the vault holds both apUSD and xBNB. The contract correctly blocks `redeem()` and `withdraw()` during this state to protect users:

SOLIDITY

```
// redeem() and withdraw() - both have this guard
if (xBNB.balanceOf(address(this)) > 0) revert Errors.UseRedeemMulti();
```

However, `deposit()` and `mint()` have **no equivalent guard**. They remain open and price shares using only the apUSD balance:

SOLIDITY

```
// deposit() - no xBNB check
function deposit(uint256 assets, address receiver) external nonReentrant whenNotPaused returns
    (uint256 shares) {
    // ... no xBNB guard here
    shares = previewDeposit(assets); // _convertToShares uses only apUSD balance
}

function _convertToShares(uint256 assets, Math.Rounding rounding) internal view returns (uint2
    56) {
    uint256 totalApUSD = apUSD.balanceOf(address(this)); // ignores xBNB
    return assets.mulDiv(supply, totalApUSD, rounding); // shares are underpriced
}
```

Since Stability Mode 2 burned apUSD from the vault, `totalApUSD` is reduced, making shares artificially cheap. However, `redeemMulti()`, which is intentionally designed to distribute both apUSD and xBNB proportionally, gives the new depositor their share of the xBNB they never paid for.

Three out of four entry points are protected. `deposit()` and `mint()` are not.

This vulnerability allows direct theft of xBNB from existing stability pool stakers. An attacker deposits apUSD at the discounted share price (which ignores xBNB), then redeems via `redeemMulti()` for both apUSD and xBNB. This is repeatable until the vault's entire xBNB balance is drained. The exploit is permissionless, requires no setup, and is executable atomically in a single block during the market stress event that Stability Mode 2 is designed to handle.

This vector can be combined with the fact that nothing prevents a user from minting as much apUSD as desired. An attacker can take a flash loan of LSTs, deposit the LSTs to obtain apUSD, reducing the CR below 130%. This would trigger Stability Mode 2, opening up the issue described earlier. The attacker would then deposit the freshly minted apUSD into the stability pool to extract profit in the form of xBNB, minus the 0.1% mint fee paid in the first step, when the CR was above 150%.

Recommendation:

For SApUSD: Apply the same guard already used on `redeem()` and `withdraw()` to `deposit()` and `mint()`, blocking deposits whenever the vault holds any xBNB:

```
function deposit(uint256 assets, address receiver) external nonReentrant whenNotPaused returns
    (uint256 shares) {
+   if (xBNB.balanceOf(address(this)) > 0) revert Errors.UseRedeemMulti();
    // ...
}

function mint(uint256 shares, address receiver) external nonReentrant whenNotPaused returns
    (uint256 assets) {
+   if (xBNB.balanceOf(address(this)) > 0) revert Errors.UseRedeemMulti();
    // ...
}
```

DIFF

Fix Review Notes:

Fixed by taking into account the value of xBNB present into the vault when pricing the shares on deposit.
The xBNB value is now resolved by the `xBNBToApUSDRate` state variable which is refreshed before every deposit/withdrawal into the vault.

M01: Inconsistent Fee Tier check timing across PoolFacet functions can create wrong incentives

Status:

Resolved

Impact:



Likelihood:



Severity:

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/PoolFacet.sol>

Description:

The dynamic fee tier system is designed with a clear incentive goal: reward actions that improve the Collateral Ratio (CR) with lower fees, and discourage actions that worsen it with higher fees.

The four core PoolFacet operations each change the collateral ratio ($CR = TVL / apUSD_supply$), and each must resolve a fee tier that depends on that CR. The timing of when the CR is checked, before or after the state-changing transfer, is inconsistent across the 4 functions.

CR effects of each operation (when $CR > 100\%$):

- **mintApUSD**: Adds equal TVL and apUSD supply - worsens the CR
- **redeemApUSD**: Removes equal TVL and apUSD supply - improves the CR
- **mintXBNB**: Adds TVL, apUSD supply unchanged - improves the CR
- **redeemXBNB**: Removes TVL, apUSD supply unchanged - worsens the CR

In the current state of the design, 3 out of 4 functions (**mintApUSD**, **redeemApUSD**, and **redeemXBNB**) check the CR before affecting the supply, while **mintXBNB** checks the CR after affecting the supply.

A user can call **mintApUSD** with a position large enough to push the protocol from a healthy CR tier into a stressed one. For instance, from Normal Mode ($CR > 150\%$) down into Stability Mode 1 ($130\% < CR < 150\%$) or even Stability Mode 2 ($CR < 130\%$), while paying fees computed against the pre-mint, healthier CR.

While it may be acceptable to charge a user the pre-action fee when their operation improves CR (effectively leaving protocol-side reward on the table), doing the same when the operation worsens CR is more dangerous. In the latter case, the pre-action fee is the more favorable one, meaning the protocol is actively undercharging for the exact behavior the tier system was designed to discourage.

Recommendation:

Perform all state-changing transfers and balance updates before reading CR and resolving the fee tier. This ensures the fee reflects the protocol state that results from the user's action, providing the intended incentive signal.

Fix Review Notes:

Fix 1: The `mintApUSD` fix is incomplete. The current version checks CR after transferring the collateral in, but before minting apUSD, although both collateral and apUSD are part of the CR computation.

This causes the CR to be inflated.

Example:

1. TVL = \$1500, supply = \$1000 (CR = 150%).

2. User deposits \$500

3. Code reads CR at: $(1500 + 500) / 1000 = 200\%$ - picks lowest fee

4. True final CR: $2000 / 1500 = 133\%$ - near Stability Mode 2

You can either use the newly developed `_getPostStateTierForMintApUSD`, or simply move the CR check after minting the apUSD.

Fix 2: The `mintApUSD` now checks the post-state CR, ensuring that users pay higher fees if the CR worsens.

M02: Malformed PancakeV3 path allows an attacker to execute arbitrary external call from the Router

Status:

Resolved

Impact:



Likelihood:



Severity:

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/periphery/AspanRouter.sol#L923-L928>

> _AspanRouter.sol

SOLIDITY

```
921:         if (pathFirstToken != swapSource) revert SwapFailed();
922:
923:         // Extract output token from path (last 20 bytes)
924:         assembly {
925:             // Load last 20 bytes from path
926:             let pathLen := path.length
927:             outputToken := shr(96, calldataload(add(path.offset, sub(pathLen, 20))))
928:         }
929:
930:         // Use Lista StableSwap for slisBNB → WBNB swaps (better rates)
```

Description:

PancakeSwap V3 encodes swap routes as a tightly packed byte sequence of the form `[token0 | fee | token1 | fee | token2 | ...]`.

Each token address occupies 20 bytes and each fee tier occupies 3 bytes (a uint24). A single-hop swap is therefore 43 bytes (20 + 3 + 20), and each additional hop adds 23 bytes (3 + 20).

In AspanRouter, `_swapFromLSTWithPath` extracts `outputToken` by reading the last 20 bytes of the user-supplied `path`.

SOLIDITY

```
assembly {
    let pathLen := path.length
    outputToken := shr(96, calldataload(add(path.offset, sub(pathLen, 20))))
}
```

It then passes this same `path` to `IPancakeV3Router.exactInput` and assumes the router received exactly `outputToken`.

This assumption breaks with a malformed 63-byte path:

```
[ swapSource(20) | fee(3) | tokenB(20) | tokenX(20) ] // 63 bytes total
```

PancakeSwap V3's `exactInput` requires `length >= 66` (see [hasMultiplePools\(\)](#)) to treat a path as multi-hop.

For anything shorter, it extracts only the first 43 bytes and performs a single-hop swap (see [swap logic](#) and [path extraction](#)), silently discarding the trailing bytes.

The Aspan Router then receives `outputAmount` of `tokenB`. Meanwhile, `_swapFromLSTWithPath` has set `outputToken = tokenX` and calls:

SOLIDITY

```
IERC20(outputToken).safeTransfer(msg.sender, outputAmount); // sends tokenX, not tokenB
```

The only path validation checks the first token only, not the path structure:

SOLIDITY

```
if (pathFirstToken != swapSource) revert SwapFailed(); // only validates first 20 bytes
```

This vulnerability also allows an attacker to:

1. Perform an external call to an arbitrary contract
2. Extract funds stuck in the router, bypassing the admin protected `rescueToken` function

This could be done by deploying a fake intermediate pool (with a fake token), constructed in a way that would allow the attacker to get the exact `outputAmount` they need. Stuck funds could result from either a user mistakenly sending funds to the router or an incomplete swap operation leaving some tokens.

Finally, the presence of the `nonReentrant` modifier and the fact that the Router has no special protocol rights lowers the impact of the arbitrary external call.

Recommendation:

Ensure that the provided `path` only accepts valid lengths:

- for a 1 pool swap, the length is 43 bytes
- for an N pool swap, the length is $43 + N * 23$ bytes

This could be verified this way:

SOLIDITY

```
if (path.length < 43 || (path.length - 43) % 23 != 0) revert InvalidPath();
```

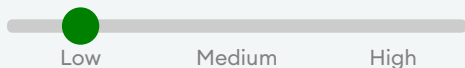
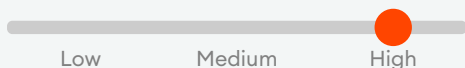
Fix Review Notes:

Fixed by implementing a new `_validateAndExtractPath` that verifies the provided path only accepts valid lengths.
The function now also ensures that `tokenOut` is allowed, as it did for `tokenIn`.

M03: 1 wei of xBNB present in the SApUSD will permanently DoS redeem() and withdraw() for all users

Status:

Resolved

Impact:**Likelihood:****Severity:**

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/tokens/SApUSD.sol#L226-L238>

>_ SApUSD.sol

SOLIDITY

```
224:      * @return assets Amount of apUSD received
225:      */
226:      function redeem(
227:          uint256 shares,
228:          address receiver,
229:          address owner
230:      ) external nonReentrant whenNotPaused returns (uint256 assets) {
231:          if (shares == 0) revert Errors.ZeroAmount();
232:          if (receiver == address(0)) revert Errors.ZeroAddress();
233:
234:          // SECURITY: Block legacy redeem when vault has xBNB to prevent loss
235:          // Users must use redeemMulti() to get all assets including xBNB
236:          if (xBNB.balanceOf(address(this)) > 0) {
237:              revert Errors.UseRedeemMulti();
238:          }
239:
240:          // Check allowance if sender is not owner
```

Description:

SApUSD.redeem() and SApUSD.withdraw() both revert if the vault holds any xBNB.

The intended goal is to protect users from accidentally bypassing xBNB compensation by using the single-asset path after a Stability Mode 2 event. However, the check relies on `xBNB.balanceOf(address(this))`, which is publicly influenceable: any account holding xBNB can transfer 1 wei directly to the SApUSD contract address at any time, and SApUSD has no mechanism to reject or account for unexpected token transfers.

Once a single wei of xBNB is present in the vault, both functions will no longer be usable. There is no recovery path: the xBNB balance cannot be cleared through `redeem()` or `withdraw()` (they revert before that), and neither function drains xBNB when it processes the withdrawal.

This means that any actor can send 1 wei of xBNB to the vault to block both functions.

It will also be an issue after a Stability Mode 2 event, as users might never be able to withdraw the full xBNB balance (all shares need to be redeemed).

Recommendation:

Switch to internal accounting to decouple the vault's logic from raw `balanceOf` calls. Introduce either a `mapping(IERC20 => uint256)` or two dedicated state variables (`accountedApUSD` and `accountedXBNB`) that track the vault's expected token balances. These internal values must be updated on every `deposit/mint/withdraw/redeem` operation.

Additionally, add a Diamond role-gated function to be called whenever apUSD or xBNB are directly minted or burned to the vault (i.e. operations that bypass the standard ERC-4626 flow):

SOLIDITY

```
enum Op { Mint, Burn }  
function updateInternalAccounting(address asset, uint256 amount, Op operation) external onlyRole(DIAMOND_ROLE);
```

All paths that directly mint or burn tokens to the vault must call this function, including: bootstrap, stability mode 2 (`triggerStabilityMode2`), and yield harvesting. Any other direct mint/burn cases that may exist should also be updated.

Optionally, add an admin `sweepExcess` function to transfer out any excess tokens sent directly to the contract (i.e. the difference between `balanceOf` and the internal accounting value), preventing griefing via unsolicited transfers.

Fix Review Notes:

Fixed according to the recommendation.

Two new variables have been added for internal accounting, preventing DoS by forced transfer:

- `accountedApUSD`
- `accountedxBNB`

Two new functions have also been added in order to account for direct mints to the vault:

- `addAccounting`
- `subAccounting`

Finally, the optional recommendation allowing admins to sweep excess tokens to remove dust tokens from the vault has been added.

- `sweepExcess`

M04: Missing slippage check in `redeemAndRequestUnstake`

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/periphery/AspanRouter.sol#L536-L540>

> **AspanRouter.sol**

SOLIDITY

```
534:
535:     // Redeem via Diamond - always redeem to slisBNB for native unstake
536:     if (redeemXBNB) {
537:         slisBNBAmount = IPoolFacet(diamond).redeemXBNB(slisBNB, amount, 0);
538:     } else {
539:         slisBNBAmount = IPoolFacet(diamond).redeemApUSD(slisBNB, amount, 0);
540:     }
541:
542:     // Request withdraw from Lista
```

Description:

`redeemAndRequestUnstake` allows users to burn apUSD or xBNB and immediately initiate a native Lista DAO unstake request (with a 7-15 day unbonding period).

Internally, it calls the PoolFacet's functions `redeemApUSD` and `redeemXBNB` with `_minOut` hardcoded to 0:

SOLIDITY

```
/// @notice Redeem apUSD for LST
/// @param _lstToken The LST token to receive
/// @param _apUSDAmount Amount of apUSD to redeem
/// @param _minOut Minimum LST to receive (slippage protection, 0 = no check)
/// @return lstAmount Amount of LST received
function redeemApUSD(
    address _lstToken,
    uint256 _apUSDAmount,
    uint256 _minOut
) external returns (uint256 lstAmount);

// ...

function redeemXBNB(
    address _lstToken,
    uint256 _xBNBAmount,
    uint256 _minOut
) external returns (uint256 lstAmount);
```

But both functions use the BNB/USD price feed to compute the amount of LST to give back to the user, which could result in the user receiving less LST than expected at the time the

transaction was sent.

All other Router functions correctly use the `_minOut` parameter.

Recommendation:

Add a `minOut` parameter to `redeemAndRequestUnstake`.

Fix Review Notes:

Fixed according to the recommendation.

M05: Missing yield harvest before collateral increase in mintApUSD/mintXBNB inflates yield for sApUSD stakers

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/PoolFacet.sol#L81-L107>

>_ PoolFacet.sol

SOLIDITY

```
79:
80:    /// @notice Mint apUSD by depositing LST
81:    /// @param _lstToken The LST token to deposit
82:    /// @param _lstAmount Amount of LST to deposit (18 decimals)
83:    /// @param _minOut Minimum apUSD to receive (slippage protection, 0 = no check)
84:    /// @return apUSDAmount Amount of apUSD minted
85:    function mintApUSD(
86:        address _lstToken,
87:        uint256 _lstAmount,
88:        uint256 _minOut
89:    ) external nonReentrant whenNotPaused returns (uint256 apUSDAmount) {
90:        if (_lstAmount == 0) revert Errors.ZeroAmount();
91:
92:        LibAppStorage.AppStorage storage s = LibAppStorage.appStorage();
93:        if (!s.lstInfo[_lstToken].isAccepted) revert Errors.LSTNotSupported();
94:        if (s.feeTiers.length == 0) revert Errors.NoFeeTiersConfigured();
95:
96:        // Get current fee tier based on CR
97:        uint256 currentCR = _getCollateralRatio();
98:        LibAppStorage.FeeTier storage tier = LibAppStorage.getFeeTierForCR(
99:            currentCR
100:        );
101:
102:        // Check if minting is disabled in this tier
103:        if (tier.apUSDMintDisabled) revert Errors.MintingDisabled();
104:
105:        // Transfer LST from user
106:        IERC20(_lstToken).safeTransferFrom(
107:            msg.sender,
108:            address(this),
109:            _lstAmount
```

Description:

`mintApUSD` and `mintXBNB` increase `collateralAmount` without first calling `_harvestAllLSTsYield()` to snapshot the current exchange rate. The redeem functions correctly harvest before touching collateral, but the mint functions do not:

SOLIDITY

```
// redeemApUSD - CORRECT: harvest first, then change collateral
_harvestAllLSTsYield(); // this one is ok
s.lstInfo[_lstToken].collateralAmount -= lstAmount;

// mintApUSD - MISSING: changes collateral without harvesting
s.lstInfo[_lstToken].collateralAmount += _lstAmount; // no harvest here
```

This matters because the yield formula multiplies `collateralAmount` by the entire rate delta since last harvest:

SOLIDITY

```
// LibYield.sol - line 124
uint256 rateIncrease = _currentExchangeRate - oldRate;
uint256 yieldBNB = Math.mulDiv(_collateralAmount, rateIncrease, PRECISION);
```

When collateral is added without harvesting first, the next harvest applies the accumulated rate increase (from before the deposit) to the inflated amount, generating phantom yield.

As a result, unbacked apUSD is minted to the sApUSD vault as phantom yield. An attacker deposits large amounts of LST without prior harvest, triggers yield distribution, and extracts inflated yield. This dilutes protocol CR and transfers value from all apUSD holders.

Recommendation:

Add `_harvestAllLSTsYield()` before the collateral update in both mint functions:

DIFF

```
function mintApUSD(...) external nonReentrant whenNotPaused returns (uint256 apUSDAmount) {
    // ... validation ...
    + _harvestAllLSTsYield();
    IERC20(_lstToken).safeTransferFrom(msg.sender, address(this), _lstAmount);
    s.lstInfo[_lstToken].collateralAmount += _lstAmount;
    // ...
}

function mintXBNB(...) external nonReentrant whenNotPaused returns (uint256 xBNBAmount) {
    // ... validation ...
    + _harvestAllLSTsYield();
    IERC20(_lstToken).safeTransferFrom(msg.sender, address(this), _lstAmount);
    s.lstInfo[_lstToken].collateralAmount += _lstAmount;
    // ...
}
```

Fix Review Notes:

Fixed according to the recommendation. Both `mintApUSD` and `mintXBNB` now call `_harvestAllLSTsYield()` before the collateral amount is updated.

M06: getLSTExchangeRate ignores useIntrinsicValue flag, breaking admin emergency recovery for redemptions

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/libraries/LibOracle.sol#L130-L138>

>_ LibOracle.sol

SOLIDITY

```
128:     * @return exchangeRate Exchange rate in 18 decimals (how much BNB per 1 LST)
129:     */
130:     function getLSTExchangeRate(address _lstToken) internal view returns (uint256 exchangeRate) {
131:         LibAppStorage.AppStorage storage s = LibAppStorage.appStorage();
132:         LibAppStorage.LSTInfo storage info = s.lstInfo[_lstToken];
133:
134:         // ONLY use intrinsic exchange rate for yield calculations
135:         // Market price fallback removed - it's not suitable for yield calculation
136:         if (info.exchangeRateProvider != address(0)) {
137:             return _getIntrinsicExchangeRate(info.exchangeRateProvider);
138:         }
139:
140:         // ■■ No exchange rate provider set - return 1:1
```

Description:

`getLSTPriceUSD()` and `getLSTExchangeRate()` handle the `useIntrinsicValue` flag inconsistently:

SOLIDITY

```
// getLSTPriceUSD - checks BOTH conditions here
if (info.useIntrinsicValue && info.exchangeRateProvider != address(0)) {
    uint256 exchangeRate = _getIntrinsicExchangeRate(info.exchangeRateProvider);
    // ...
}
// Falls through to Chainlink if useIntrinsicValue == false

// getLSTExchangeRate - checks only provider address
if (info.exchangeRateProvider != address(0)) {
    return _getIntrinsicExchangeRate(info.exchangeRateProvider);
}
// Never falls through when provider is set, regardless of flag
```

Now when an exchange rate provider breaks (paused, reverting, out-of-bounds), the admin's expected emergency action is:

SOLIDITY

```
setLSTExchangeRateProvider(lstToken, existingProvider, false);  
// Sets useIntrinsicValue = false - "switch to Chainlink pricing"
```

After this, `getLSTPriceUSD()` correctly skips the broken provider, and minting resumes. But `getLSTExchangeRate()` ignores the flag and still calls the broken provider. Yield harvest continues reverting.

Since `redeemApUSD()` and `redeemXBNB()` both call `_harvestAllLSTsYield()` before executing, and the harvest loop calls `getLSTExchangeRate()` for every LST without error isolation, **all redemptions remain frozen** even after the admin believes they have resolved the outage.

One clear concrete impact of this is that the admin emergency switch only partially restores operations. Minting works, but all redemptions stay blocked. The admin must discover and perform a non-obvious second step (zeroing out the provider address entirely) to unblock redemptions, extending protocol downtime during a provider failure.

Recommendation:

Add the missing `useIntrinsicValue` check to match `getLSTPriceUSD`:

DIFF

```
function getLSTExchangeRate(address _lstToken) internal view returns (uint256 exchangeRate) {  
    LibAppStorage.AppStorage storage s = LibAppStorage.appStorage();  
    LibAppStorage.LSTInfo storage info = s.lstInfo[_lstToken];  
-   if (info.exchangeRateProvider != address(0)) {  
+   if (info.useIntrinsicValue && info.exchangeRateProvider != address(0)) {  
        return _getIntrinsicExchangeRate(info.exchangeRateProvider);  
    }  
    return PRECISION;  
}
```

Fix Review Notes:

Fixed according to the recommendation.

M07: MEV protection bypass via direct SApUSD vault access

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/tokens/SApUSD.sol#L134>

> _SApUSD.sol

SOLIDITY

```
132:      * @return shares Amount of shares minted
133:      */
134:      function deposit(
135:          uint256 assets,
136:          address receiver
```

Description:

`StabilityPoolFacet` enforces a `minDepositBlocks` lock (default 3 blocks) to prevent yield sandwiching:

SOLIDITY

```
// StabilityPoolFacet.deposit() - records block
LibAppStorage.recordDeposit(msg.sender);
// StabilityPoolFacet.withdraw() - enforces lock
LibAppStorage.enforceDepositLock(msg.sender);
```

But the underlying SApUSD vault's `deposit()`, `mint()`, `redeem()`, `withdraw()`, and `redeemMulti()` are all fully public. There is no `DIAMOND_ROLE` check and no lock. This means anyone can call them directly, bypassing the Diamond entirely.

The `DIAMOND_ROLE` already exists and is granted via `approveDiamond()`. It just is not used to gate these functions.

As a result, an attacker can sandwich yield harvests in a single block: deposit before harvest, redeem after, stealing accumulated yield from legitimate stakers. The `minDepositBlocks` MEV protection is entirely ineffective.

Recommendation:

Gate vault entry points with the existing `DIAMOND_ROLE`:

DIFF

```
+ modifier onlyDiamond() {
+     if (!hasRole(DIAMOND_ROLE, msg.sender)) revert Errors.Unauthorized();
+     _;
+ }
+
function deposit(...)    external onlyDiamond nonReentrant whenNotPaused { ... }
```

... continued

DIFF

```
function redeem(...)    external onlyDiamond nonReentrant whenNotPaused { ... }  
function withdraw(...) external onlyDiamond nonReentrant whenNotPaused { ... }  
function redeemMulti(...) external onlyDiamond nonReentrant whenNotPaused { ... }
```

Fix Review Notes:

Fixed according to the recommendation.
The functions now have `onlyRole(DIAMOND_ROLE)` access control.

M08: Unbounded array copy in `_requestListaWithdraw` causes progressive DoS of native unstake path

Status:

Resolved

Impact:



Likelihood:



Severity:

Medium

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/periphery/AspanRouter.sol#L584-L591>

> `_AspanRouter.sol`

SOLIDITY

```
582:
583:     /// @notice Internal function to request withdrawal from Lista
584:     function _requestListaWithdraw(uint256 slisBNBAmount) internal returns (uint256 requestIndex) {
585:         // Approve Lista to spend slisBNB
586:         IERC20(slisBNB).approve(listaStakeManager, slisBNBAmount);
587:
588:         // Get current number of requests to determine the new index
589:         IListaStakeManager.WithdrawalRequest[] memory existingRequests =
590:             IListaStakeManager(listaStakeManager).getUserWithdrawalRequests(address(this));
591:         uint256 listaIdx = existingRequests.length;
592:
593:         // Request withdraw from Lista
```

Description:

To determine the next withdrawal index, the router copies Lista's entire withdrawal request array into memory just to read its length:

SOLIDITY

```
IListaStakeManager.WithdrawalRequest[] memory existingRequests =
    IListaStakeManager(listaStakeManager).getUserWithdrawalRequests(address(this));
uint256 listaIdx = existingRequests.length;
```

All users' unstake requests go through the router as `msg.sender`, so this array grows monotonically in Lista's storage and never shrinks (claimed requests persist). Each call to `redeemAndRequestUnstake` adds one entry. The memory copy cost scales $O(n)$ with every past request.

An attacker can accelerate this by calling `redeemAndRequestUnstake` with dust amounts (1 wei of apUSD) repeatedly. After approximately 50k to 100k entries, the `returndatacopy` exceeds the block gas limit and the function permanently reverts.

As a result, `redeemAndRequestUnstake()` becomes permanently bricked for all users, killing the router's only zero-slippage native BNB exit path. Users are forced into DEX swaps with slippage as the only alternative.

Recommendation:

The full fix is to eliminate `getUserWithdrawalRequests()` entirely. For UUID retrieval in `_requestListaWithdraw`, check if Lista exposes a `nextWithdrawId` counter or a single-element accessor. If so, use it to avoid the array copy. Otherwise, consider caching the UUID at request time by comparing the array length delta, or adding `nextWithdrawId()` to the `IListaStakeManager` interface if the on-chain contract supports it.

Fix 2: Both `getUserWithdrawalRequests()` calls have been fully eliminated. Index tracking now uses a local `nextListIdx` counter and bidirectional `uuidToListIdxPlusOne/listIdxToUUID` mappings, making both `_requestListaWithdraw` and `claimUnstake` $O(1)$ regardless of history size. Issue fully resolved.

L01: Absence of oracle fallback mechanism creates single point of failure for BNB price dependency

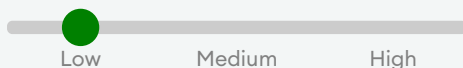
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/libraries/LibOracle.sol#L41-L69>

>_ LibOracle.sol

SOLIDITY

```
39:         if (s.bnbPriceFeed == address(0)) revert Errors.ZeroAddress();
40:
41:         IAggregatorV3 priceFeed = IAggregatorV3(s.bnbPriceFeed);
42:
43:         (
44:             uint80 roundId,
45:             int256 answer,
46:             ,
47:             uint256 updatedAt,
48:             uint80 answeredInRound
49:         ) = priceFeed.latestRoundData();
50:
51:         // ■ Complete Chainlink safety checks
52:         if (answer <= 0) revert Errors.InvalidPrice();
53:         if (answeredInRound < roundId) revert Errors.StalePrice();
54:         if (updatedAt == 0) revert Errors.InvalidPrice();
55:         if (block.timestamp - updatedAt > s.maxPriceAge) revert Errors.StalePrice();
56:
57:         priceUSD = uint256(answer); // Safe conversion, already checked > 0
58:
59:         // Validate price is within configured bounds
60:         LibAppStorage.OracleBounds memory bounds = s.oracleBounds[s.bnbPriceFeed];
61:         if (bounds.minPrice > 0 && priceUSD < bounds.minPrice) {
62:             revert Errors.PriceBelowMinimum();
63:         }
64:         if (bounds.maxPrice > 0 && priceUSD > bounds.maxPrice) {
65:             revert Errors.PriceAboveMaximum();
66:         }
67:
68:         return priceUSD;
69:     }
70:
71:     /**
```

Description:

`LibOracle.getBNBPriceUSD()` reads from a single Chainlink AggregatorV3 feed. The function implements standard validation checks (positive answer, round completeness, staleness via `maxPriceAge`, and configurable price bounds), all of which revert on failure.

There is no secondary oracle or TWAP fallback allowing the protocol to continue functioning, although the BNB/USD price is a dependency for a wide portion of the protocol:

- xBNB operations: `mintXBNB()` and `redeemXBNB()` directly call `getBNBPriceUSD()` for price conversions.
- Stability pool: Every `deposit()`, `withdraw()`, and `withdrawAssets()` triggers yield harvesting, which calls `getBNBPriceUSD()`. This means users cannot exit the stability pool during an oracle outage.
- Intrinsic-value LSTs: For LSTs configured with `useIntrinsicValue = true` (`slisBNB`, `wclisBNB`), even `mintApUSD()` and `redeemApUSD()` depend on the BNB feed, since intrinsic pricing computes `exchangeRate * bnbPriceUSD`.

Recommendation:

Introduce a fallback mechanism for the BNB/USD price. This could be either a secondary Chainlink price feed, or TWAP from a highly liquid BNB/USDT trading pool.

Fix Review Notes:

Fixed according to the recommendation. A secondary fallback oracle has now been added.

L02: Single global maxPriceAge exposes the protocol to stale prices, leading to DoS or value extraction

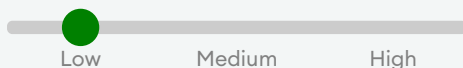
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/libraries/LibAppStorage.sol#L79-L80>

>_ LibAppStorage.sol

SOLIDITY

```
77:      uint256 protocolFeesBNB; // Accumulated protocol fees in BNB terms
78:      bool paused;
79:      // ===== Oracle Config =====
80:      uint256 maxPriceAge; // Max price staleness
81:
82:      mapping(address => OracleBounds) oracleBounds; // PriceFeed => bounds
```

Description:

`LibAppStorage.AppStorage` stores a single uint256 `maxPriceAge` value.

This single value is applied uniformly to every Chainlink price check in the protocol, both the BNB/USD feed and every LST/USD feed.

SOLIDITY

```
// LibOracle.sol:55 - BNB price feed
if (block.timestamp - updatedAt > s.maxPriceAge) revert Errors.StalePrice();

// LibOracle.sol:201 - LST price feed
if (block.timestamp - updatedAt > s.maxPriceAge) revert Errors.StalePrice();
```

Chainlink feeds have different heartbeat intervals (the maximum time between on-chain updates). On BSC, for example:

- BNB/USD has a 27-second heartbeat (updates on every 0.1% deviation or 27s elapsed).
- LST/USD feeds (e.g., slisBNB/USD) would typically operate on a longer heartbeat based on the current state of existing Chainlink price feeds.

Then, if the `maxPriceAge` is configured using the BNB/USD price feed, access to the LST/USD price would revert after 27 seconds, even though the price is still valid according to its heartbeat. If `maxPriceAge` is instead configured based on the LST/USD price feed, this will make it possible to read outdated prices, allowing users to mint/redeem apUSD/xBNB at better (or worse) prices than they should.

Recommendation:

Replace the single global `maxPriceAge` with a per-feed mapping.

Fix Review Notes:

Fixed according to the recommendation. A `maxPriceAgeByFeed` mapping and `_maxPriceAgeForFeed` function were added.

L03: apUSD minting and xBNB redeeming lacks post-state CR enabling forced conversion of the stability pool

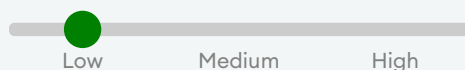
Status:

Acknowledged

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/PoolFacet.sol#L97-L125>

>_ PoolFacet.sol

SOLIDITY

```
95:
96:     // Get current fee tier based on CR
97:     uint256 currentCR = _getCollateralRatio();
98:     LibAppStorage.FeeTier storage tier = LibAppStorage.getFeeTierForCR(
99:         currentCR
100:    );
101:
102:     // Check if minting is disabled in this tier
103:     if (tier.apUSDMintDisabled) revert Errors.MintingDisabled();
104:
105:     // Transfer LST from user
106:     IERC20(_lstToken).safeTransferFrom(
107:         msg.sender,
108:         address(this),
109:         _lstAmount
110:    );
111:     s.lstInfo[_lstToken].collateralAmount += _lstAmount;
112:
113:     // Calculate USD value with proper decimals handling
114:     uint256 lstPriceUSD = _getLSTPriceUSD(_lstToken);
115:     uint256 valueUSD = (_normalizeToDecimals18(_lstAmount, s.lstInfo[_lstToken].de
116: cimals) * lstPriceUSD) / PRECISION;
117:
118:     // Apply fee from current tier
119:     uint256 fee = (valueUSD * tier.apUSDMintFee) / BPS;
120:     apUSDAmount = valueUSD - fee;
121:
122:     // Slippage protection
123:     if (_minOut > 0 && apUSDAmount < _minOut) revert Errors.SlippageExceeded();
124:
125:     // Mint apUSD to user
126:     _mintApUSD(msg.sender, apUSDAmount);
127:
128:     // Mint fee to treasury
```

Description:

(The following report focuses on mintApUSD, but the same reasoning can be applied to redeemXBNB)

mintApUSD evaluates the collateral ratio (CR) before the deposit and mint occur. The result is used to select a fee tier and check the `apUSDMintDisabled` flag, but neither the fee tier nor any other check is re-evaluated after the state change.

CR is computed as TVL of all LSTs divided by the supply of apUSD ($CR = \text{tvUSD} / \text{apUSD_supply}$). Because $CR > 100\%$ implies $TVL > \text{supply}$, adding the same value V to both numerator and denominator always reduces CR, converging it toward 100%. A single mint large enough relative to the pool can cross the 130% Mode 2 boundary in one transaction.

Once CR drops below the Stability Mode 2 threshold (130%), anyone can call the public function `triggerStabilityMode2()`. This burns apUSD directly from the sApUSD Stability Pool and mints an equivalent value of xBNB to it in order to increase the CR, at the expense of sApUSD holders.

If a user holds a substantial amount of LST and mints apUSD, this would cause the CR to cross 130%. Then, an on-chain observer calls `triggerStabilityMode2()`, converting apUSD from the stability pool to xBNB. sApUSD holders are now forcibly exposed to xBNB, which has a higher redeeming fee compared to apUSD.

Additionally, exiting through `redeemxBNB` has 8% protocol fees, so we can imagine that users many will prefer selling on the secondary market, creating downward price pressure. An attacker with a pre-established xBNB short would then profit from this.

Recommendation:

1. Enforce a post-mint CR floor with a safety buffer:

```
SOLIDITY
function mintApUSD(...) external nonReentrant whenNotPaused returns (uint256 apUSDAmount) {
    // ... existing logic ...
    _mintApUSD(msg.sender, apUSDAmount);
    if (fee > 0) _mintApUSD(s.treasury, fee);

    // Ensure the mint did not push CR close to Stability Mode 2
    if (_getCollateralRatio() < MINIMUM_MINT_CR) revert Errors.MintWouldBreachCRFloor();
}
```

`MINIMUM_MINT_CR` should be above the Mode 2 threshold with a safety buffer.

2. Additionally, consider determining the fee tier based on the post-mint collateral ratio, especially when the user action worsens the system CR.

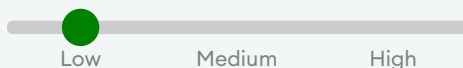
Fix Review Notes:

Acknowledged. The team considered the risk acceptable now that operations that worsen the CR use the post-state CR.

L04: Deferred yield minting in `_harvestYield` still enables first-depositor capture of accumulated yield

Status:

Resolved

Impact:**Likelihood:****Severity:**

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/StabilityPoolFacet.sol#L250-L272>

>_ StabilityPoolFacet.sol**SOLIDITY**

```
248:      * - Accumulated yield will be distributed when pool has depositors
249:      */
250:      function _harvestYield() internal {
251:          LibAppStorage.AppStorage storage s = LibAppStorage.appStorage();
252:
253:          // 1. Calculate new yield from all LSTs (returns total, doesn't accumulate)
254:          uint256 newYieldUSD = _calculateAndRecordYield();
255:
256:          // 2. Accumulate the new yield to pending
257:          if (newYieldUSD > 0) {
258:              LibYield.accumulatePendingYield(newYieldUSD);
259:          }
260:
261:          // 3. Get total pending yield (includes both new and historical)
262:          uint256 totalPending = LibYield.getPendingYield();
263:
264:          if (totalPending > 0 && s.sApUSD != address(0)) {
265:              // Check if stability pool has any deposits
266:              uint256 totalAssets = IERC4626(s.sApUSD).totalAssets();
267:
268:              if (totalAssets == 0) {
269:                  // Pool is empty - keep all pending yield for later distribution
270:                  // This prevents first depositor from capturing all accumulated yield
271:                  return;
272:              }
273:
274:              // Pool has deposits - mint ALL pending yield to vault
```

Description:

The `_harvestYield()` function contains a logic flaw in how it handles yield accumulation when the stability pool has no depositors.

When the stability pool is empty, `_harvestYield()` checks `totalAssets == 0` and returns early, leaving `pendingYield` unminted. Since `_harvestYield()` runs before the ERC-4626 `deposit()` call, the first depositor receives shares at the 1:1 rate. On the next interaction where `totalAssets > 0`, all accumulated `pendingYield` is minted via `consumePendingYield()`, inflating the value of existing shares, which belong entirely to that first depositor.

The code comment states this "prevents first depositor from capturing all accumulated yield", but in reality this simply defers the capture.

Additionally, the existing deposit lock `enforceDepositLock` does not mitigate this either. It checks `lastDepositBlock[msg.sender]`, but `sApUSD` is a freely transferable ERC-20. Transferring shares to a fresh address bypasses the lock entirely. There is no other mechanism preventing same-block extraction of the accumulated yield.

An attacker could simply:

1. Deposit 1 wei of `ApUSD` (which will not distribute the yield to the pool due to the current implementation)
2. Transfer the share to a secondary address
3. Call `withdraw` with the second address, which will pass the deposit lock, and this time, distribute the yield to the pool
4. Burn the share in exchange for the freshly distributed yield

However, the likelihood of such an event is quite low, which reduces the overall severity of the issue.

Recommendation:

An option would be to redirect yield that accrues while the pool is empty to a protocol-controlled reserve:

```
if (totalAssets == 0) {
    uint256 newYieldUSD = _calculateAndRecordYield();
    if (newYieldUSD > 0) {
        IApUSD(s.apUSD).mint(s.protocolReserve, newYieldUSD);
    }
    return;
}
```

SOLIDITY

The reserve doubles as an emergency fund during Stability Mode, from where the protocol would draw first before converting staker deposits. This will be beneficial to all stakers whenever the mode is activated, by delaying the forced swap.

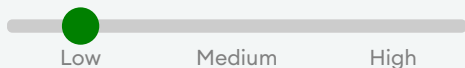
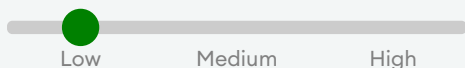
Fix Review Notes:

Fixed according to the recommendation. Yield is now redirected to protocol treasury while the pool is empty.

L05: setLSTExchangeRateProvider does not reinitialize yield tracking, causing phantom or frozen yield

Status:

Resolved

Impact:**Likelihood:****Severity:**

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/OracleFacet.sol#L154-L171>

>_ OracleFacet.sol

SOLIDITY

```
152:    /// @param _provider Exchange rate provider contract (implements ILSTExchangeRate)
153:    /// @param _useIntrinsicValue Whether to use intrinsic value for pricing
154:    function setLSTExchangeRateProvider(
155:        address _lstToken,
156:        address _provider,
157:        bool _useIntrinsicValue
158:    ) external onlyOwner {
159:        LibAppStorage.AppStorage storage s = LibAppStorage.appStorage();
160:        if (!s.lstInfo[_lstToken].isAccepted) revert Errors.LSTNotFound();
161:
162:        // If enabling intrinsic value, provider must be set
163:        if (_useIntrinsicValue && _provider == address(0)) {
164:            revert Errors.ZeroAddress();
165:        }
166:
167:        s.lstInfo[_lstToken].exchangeRateProvider = _provider;
168:        s.lstInfo[_lstToken].useIntrinsicValue = _useIntrinsicValue;
169:
170:        emit LSTExchangeRateProviderSet(_lstToken, _provider, _useIntrinsicValue);
171:    }
172:
173:    /// @notice Update LST price feed address
```

Description:

When the admin swaps an LST's exchange rate provider via `setLSTExchangeRateProvider()`, the function updates the provider pointer but does not reset the `lastExchangeRate` stored in `LibYield`. This value still holds the last reading from the old provider.

On the next harvest, `LibYield.calculateAndRecordYield()` compares the new provider's rate against the old provider's stale snapshot:

SOLIDITY

```
uint256 oldRate = info.lastExchangeRate; // stale value from old provider
// ...
if (_currentExchangeRate > oldRate && oldRate > 0 && _collateralAmount > 0) {
    uint256 rateIncrease = _currentExchangeRate - oldRate; // cross-provider delta
    uint256 yieldBNB = Math.mulDiv(_collateralAmount, rateIncrease, PRECISION);
```

This produces one of two incorrect outcomes:

- new rate > old rate: Phantom yield is minted as apUSD to the sApUSD vault, inflating share value with unbacked tokens
- new rate < old rate: Yield freezes entirely until the new provider's rate exceeds the stale threshold

Compare with `addLSTWithProvider()` which correctly calls `LibYield.initLSTYieldTracking(_lstToken, initialExchangeRate). setLSTExchangeRateProvider()` omits this step.

A manual workaround exists (`YieldFacet.initLSTYield()`), but it requires a separate transaction and is not enforced. If a harvest executes between the provider swap and the manual reset (via `StabilityPoolFacet.deposit/withdraw` or `harvestYield()`), phantom yield is minted.

Recommendation:

Reinitialize yield tracking atomically when the provider changes:

```
function setLSTExchangeRateProvider(
    address _lstToken, address _provider, bool _useIntrinsicValue
) external onlyOwner {
    // ... existing validation ...
    s.lstInfo[_lstToken].exchangeRateProvider = _provider;
    s.lstInfo[_lstToken].useIntrinsicValue = _useIntrinsicValue;

+   if (_provider != address(0)) {
+       uint256 currentRate = ILSTExchangeRate(_provider).getExchangeRate();
+       LibYield.initLSTYieldTracking(_lstToken, currentRate);
+   }

    emit LSTExchangeRateProviderSet(_lstToken, _provider, _useIntrinsicValue);
}
```

DIFF

Fix Review Notes:

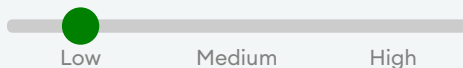
Fixed according to the recommendation. The yield tracker is now reset.

L06: cleanXbnb() selector not registered in diamond proxy, protocol recovery function unreachable

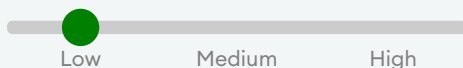
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/script/DeployBSC.s.sol#L309>

>_ DeployBSC.s.sol

SOLIDITY

```
307: // ===== Facet Cuts Builder =====
308:
309: function _buildFacetCuts() internal view returns (IDiamondCut.FacetCut[] memory cuts) {
310:     cuts = new IDiamondCut.FacetCut[](8);
311:
```

Description:

`PoolFacet.cleanXbnb()` allows users to burn worthless xBNB when the token is underwater (TVL $\leq$ apUSD supply, xBNB price = 0). This is the protocol's recovery mechanism for extreme BNB price drops.

The function exists in `PoolFacet.sol` but its selector is not included in the `poolSelectors` array inside `_buildFacetCuts()`. The array registers 14 selectors (indices 0-13, `mintApUSD`, `redeemApUSD`, `mintXBNB`, `redeemXBNB`, and 10 view functions) but omits `cleanXbnb`. Since the Diamond proxy only routes calls to selectors explicitly registered via `diamondCut`, any call to `cleanXbnb()` through the Diamond will hit the fallback and revert.

Now when BNB drops enough to make xBNB worthless (price = 0), the protocol enters a degraded state, `mintXBNB` reverts (`XBNBUnderwater`), and `redeemXBNB` returns nothing. `cleanXbnb` is the only path to reduce xBNB supply and allow the protocol to recover once prices stabilize. Without it, the xBNB supply remains inflated permanently, and the protocol cannot organically restore a healthy collateral ratio. This would require an emergency Diamond cut to register the missing selector under duress.

Recommendation:

Add the `cleanXbnb` selector to the `poolSelectors` array in `_buildFacetCuts()`:

```
// Change array size from 14 to 15
bytes4[] memory poolSelectors = new bytes4[](15);
// ... existing 14 selectors ...
poolSelectors[14] = PoolFacet.cleanXbnb.selector;
```

SOLIDITY

If already deployed, execute a `diamondCut` to add the missing selector to the existing `PoolFacet` address. No new facet deployment is needed, the implementation is already on-chain, it just needs to be wired up.

Fix Review Notes:

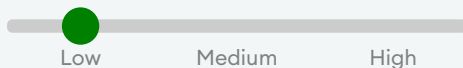
Fixed according to the recommendation. The array has been resized, and `cleanXbnb.selector` added at index 4.

L07: `mintXBNB` does not enforce bootstrap, allowing first minter to capture existing equity at hardcoded 1 bnb price

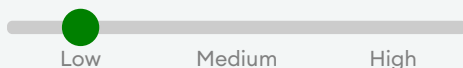
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/PoolFacet.sol#L211>

> `PoolFacet.sol`

SOLIDITY

```
209:    /// @param _minOut Minimum xBNB to receive (slippage protection, 0 = no check)
210:    /// @return xBNBAmount Amount of xBNB minted
211:    function mintXBNB(
212:        address _lstToken,
213:        uint256 _lstAmount,
```

Description:

When `xBNB.totalSupply() == 0`, `_getXBNBPriceBNB()` returns a hardcoded fallback price of 1 BNB:

SOLIDITY

```
uint256 xBNBSupply = IERC20(s.xBNB).totalSupply();
if (xBNBSupply == 0) return PRECISION; // 1 BNB
```

`ConfigFacet.bootstrap()` exists to prevent this by minting xBNB to a dead address before normal operations begin. However, `mintXBNB()` does not check `s.bootstraped`, so if any apUSD is minted before bootstrap is called, an attacker can mint xBNB at 1 BNB while the real backing per xBNB is far higher.

The first xBNB minter captures the accumulated equity buffer (TVL minus apUSD liabilities) at a fixed 1 BNB price instead of the true backing value. This is a one-time launch window issue. Once `bootstrap()` is called, the xBNB supply is permanently nonzero and the window closes.

Recommendation:

Enforce bootstrap before allowing xBNB minting:

DIFF

```
function mintXBNB(...) external nonReentrant whenNotPaused returns (uint256 xBNBAmount) {
+   if (!s.bootstraped) revert Errors.NotBootstrapped();
    // ...
}
```

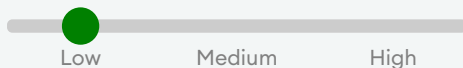
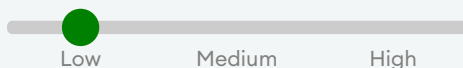
Fix Review Notes:

Fixed according to the recommendation.

L08: `_getXBNBPriceBNB` does not subtract `pendingYieldUSD` from `xBNB` backing, allowing `xBNB` redeemers to extract yield owed to `sApUSD` stakers

Status:

Resolved

Impact:**Likelihood:****Severity:**

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/PoolFacet.sol#L526-L546>

```
>_ PoolFacet.sol SOLIDITY
524:     }
525:
526:     function _getXBNBPriceBNB() internal view returns (uint256) {
527:         LibAppStorage.AppStorage storage s = LibAppStorage.appStorage();
528:
529:         uint256 xBNBSupply = IERC20(s.xBNB).totalSupply();
530:         if (xBNBSupply == 0) return PRECISION; // Initial price = 1 BNB
531:
532:         uint256 tvlBNB = _getTVLInBNB();
533:         uint256 apUSDSupply = IERC20(s.apUSD).totalSupply();
534:         uint256 bnbPriceUSD = _getBNBPriceUSD();
535:
536:         // Convert apUSD supply to BNB
537:         uint256 apUSDValueBNB = (apUSDSupply * CHAINLINK_DECIMALS) /
538:             bnbPriceUSD;
539:
540:         // xBNB backing = TVL - apUSD value
541:         if (tvlBNB <= apUSDValueBNB) return 0; // Underwater
542:         uint256 xBNBBackingBNB = tvlBNB - apUSDValueBNB;
543:
544:         // xBNB price = backing / supply
545:         return (xBNBBackingBNB * PRECISION) / xBNBSupply;
546:     }
547:
548:     function _getXBNBPriceUSD() internal view returns (uint256) {
```

Description:

Per the code comments, 100% of LST staking yield belongs to `sApUSD` stakers. `xBNB` holders receive leveraged price exposure only, no yield.

Yield distribution happens in two phases:

1. `PoolFacet` harvests yield and records it in `pendingYieldUSD` (a promise to `sApUSD` stakers).
2. `StabilityPoolFacet` later mints that pending amount as `apUSD` to the `sApUSD` vault (fulfilling the promise).

Now between these two phases, `_getXBNBPriceBNB` misprices `xBNB`. TVL already reflects the yield (via live exchange rates) while `apUSDSupply` has not yet been increased by the pending

yield mint:

SOLIDITY

```
function _getXBNBPriceBNB() internal view returns (uint256) {
    uint256 tvlBNB = _getTVLInBNB(); // includes yield
    uint256 apUSDSupply = IERC20(s.apUSD).totalSupply(); // excludes pending yield
    uint256 apUSDValueBNB = (apUSDSupply * CHAINLINK_DECIMALS) / bnbPriceUSD;
    uint256 xBNBBackingBNB = tvlBNB - apUSDValueBNB; // inflated
    return (xBNBBackingBNB * PRECISION) / xBNBSupply;
}
```

`pendingYieldUSD` is never read or subtracted. This is directly exploitable in `redeemXBNB`, which first harvests (increasing `pendingYieldUSD`) then prices xBNB using the inflated backing:

```
redeemXBNB()
→ _harvestAllLSTsYield() // pendingYieldUSD += yield
→ _calculateLSTAmountForXBNB() // uses _getXBNBPriceBNB(), does not subtract pending
→ user receives more LST than entitled
```

The same inflation affects `mintXBNB`, where users receive fewer xBNB tokens than they should since the buy price is inflated.

One concrete impact of this is that xBNB redeemers extract LST collateral backed by yield owed to sApUSD stakers. This systematically transfers value from sApUSD stakers to xBNB redeemers on every redemption where pending yield exists.

Recommendation:

Subtract `pendingYieldUSD` from xBNB backing by treating it as an apUSD liability:

DIFF

```
function _getXBNBPriceBNB() internal view returns (uint256) {
    // ...
    uint256 tvlBNB = _getTVLInBNB();
    uint256 apUSDSupply = IERC20(s.apUSD).totalSupply();
    uint256 bnbPriceUSD = _getBNBPriceUSD();

+   uint256 pendingYield = LibYield.getPendingYield();
+   uint256 effectiveApUSDSupply = apUSDSupply + pendingYield;

-   uint256 apUSDValueBNB = (apUSDSupply * CHAINLINK_DECIMALS) / bnbPriceUSD;
+   uint256 apUSDValueBNB = (effectiveApUSDSupply * CHAINLINK_DECIMALS) / bnbPriceUSD;

    if (tvlBNB <= apUSDValueBNB) return 0;
    uint256 xBNBBackingBNB = tvlBNB - apUSDValueBNB;

    return (xBNBBackingBNB * PRECISION) / xBNBSupply;
}
```

Fix Review Notes:

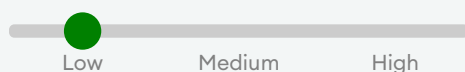
Fixed according to the recommendation. Pending yield is now treated as an apUSD liability.

L09: Excess msg.value silently accepted and stuck in router

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/periphery/AspanRouter.sol#L683-L688>

> _AspanRouter.sol

SOLIDITY

```
681:     * @dev Wraps native BNB to WBNB if needed
682:     */
683:     function _receiveInputTokens(address token, uint256 amount) internal {
684:         if (token == address(0)) {
685:             // Native BNB - already received via msg.value
686:             if (msg.value < amount) revert InsufficientOutput();
687:             // Wrap to WBNB for swap
688:             IWBNB(wbnb).deposit{value: amount}();
689:         } else {
690:             // ERC20 token
```

Description:

`_receiveInputTokens` has two vectors where user BNB gets stuck in the Router:

SOLIDITY

```
function _receiveInputTokens(address token, uint256 amount) internal {
    if (token == address(0)) {
        if (msg.value < amount) revert InsufficientOutput(); // allows msg.value > amount
        IWBNB(wbnb).deposit{value: amount}();                // only wraps `amount`, excess stays
    } else {
        IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
        // no msg.value check, BNB sent alongside ERC20 is silently kept
    }
}
```

Vector 1, the native BNB path: The check is `<` not `!=`. If a user sends more BNB than `amount`, only `amount` is wrapped. The excess remains as raw BNB in the Router with no return path.

Vector 2, the ERC20 path: The `swapAndMint` function is `payable`, so the ERC20 branch can receive BNB. The code never checks `msg.value == 0` when `token != address(0)`. Any BNB accidentally sent alongside a USDT/USDC swap is silently accepted and stuck.

This is inconsistent with the Router's own `swapAndMintDefault` function, which calls `_swapToLSTSimple` and correctly uses strict equality (`msg.value != inputAmount`). This confirms the lax check is unintentional.

As a result, users lose excess or accidentally-sent BNB. Funds are only recoverable via admin calling `rescueTokens()`, which makes the impact lower.

Recommendation:

Add strict equality on the native path and zero-check on the ERC20 path:

DIFF

```
function _receiveInputTokens(address token, uint256 amount) internal {  
    if (token == address(0)) {  
-        if (msg.value < amount) revert InsufficientOutput();  
+        if (msg.value != amount) revert IncorrectBNBAmount();  
        IWBNB(wbnb).deposit{value: amount}();  
    } else {  
+        if (msg.value != 0) revert UnexpectedBNB();  
        IERC20(token).safeTransferFrom(msg.sender, address(this), amount);  
    }  
}
```

Fix Review Notes:

Fixed according to the recommendation.

5. Enhancement Opportunities

E01: AspanRouter does not cover all four PancakeSwap V3 Fee Tiers

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/periphery/AspanRouter.sol#L233-L242>

```
>_ AspanRouter.sol SOLIDITY
231:     }
232:
233:     modifier validateSwapParams(SwapParams calldata params) {
234:         if (!supportedInputTokens[params.inputToken]) revert UnsupportedInputToken();
235:
236:         address targetLST = params.targetLST == address(0) ? defaultLST : params.targetLST;
237:         if (!supportedLSTs[targetLST]) revert UnsupportedLST();
238:
239:         if (params.inputAmount == 0) revert ZeroAmount();
240:         if (params.poolFee != 500 && params.poolFee != 2500 && params.poolFee != 10000)
241:         ) {
242:             revert InvalidPoolFee();
243:         }
244:     }
```

Description:

The `validateSwapParams` modifier in `AspanRouter` enforces a whitelist of accepted PancakeSwap V3 fee tiers. This covers fee tiers of 0.05% (500), 0.25% (2500), and 1% (10000). However, PancakeSwap V3 on BSC supports 4 fee tiers.

The 0.01% tier is notably the primary tier for highly correlated or pegged asset pairs, which are exactly the input tokens this router is designed to handle.

Potential Benefit:

Give integrators and users full routing flexibility across all live PancakeSwap V3 pools.

Recommendation:

Add 100 to the fee tier allowlist in `validateSwapParams`, and update the interface `NatSpec` accordingly:

```
// AspanRouter.sol:240 DIFF
- if (params.poolFee != 500 && params.poolFee != 2500 && params.poolFee != 10000) {
+ if (params.poolFee != 100 && params.poolFee != 500 && params.poolFee != 2500 &&
+   params.poolFee != 10000) {
    revert InvalidPoolFee();
}
```

```
// IAspanRouter.sol:18 DIFF
```

... continued

DIFF

```
- uint24 poolFee; // PancakeSwap V3 pool fee tier (500, 2500, 10000)
+ uint24 poolFee; // PancakeSwap V3 pool fee tier (100, 500, 2500, 10000)
```

Fix Review Notes:

Acknowledged.

E02: Yield calculation uses raw `collateralAmount` without decimal normalization, causing incorrect yield for non-18-decimal LSTs

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/src/diamond/facets/PoolFacet.sol#L633>

```
>_ PoolFacet.sol SOLIDITY
631:         // If mint fails, it will automatically revert with the actual error
632:     }
633:
634:
635:     function _burnXBNB(address _from, uint256 _amount) internal {
```

Description:

`LibYield.calculateAndRecordYield()` documents its `_collateralAmount` parameter as "Amount of LST in pool (18 decimals)" and computes yield as `collateralAmount * rateIncrease / 1e18`. All callers pass `lstInfo.collateralAmount` directly, the raw amount stored in the LST's native decimals without calling `_normalizeToDecimals18()`. Every other value calculation in the protocol (TVL, mint, redeem, xBNB pricing) normalizes via `_normalizeToDecimals18(collateralAmount, decimals)` before arithmetic, but the yield path does not.

Currently all supported LSTs (slisBNB, asBNB, wclisBNB) are 18 decimals, so the raw value equals the normalized value and yield is correct. However, the protocol's `addLST()` and `addLSTWithProvider()` accept an arbitrary `uint8 _decimals` parameter and `LSTInfo` stores it. The architecture explicitly supports non-18-decimal tokens. If an admin adds an 8-decimal LST, yield for that token would be underreported by a factor of 10^{10} (effectively zero yield captured), silently breaking yield distribution to sApUSD stakers.

Potential Benefit:

Normalizing `collateralAmount` to 18 decimals before yield calculation ensures yield is correctly computed regardless of LST decimals. This future-proofs the protocol against silent yield loss when onboarding new LSTs, maintaining consistency with how every other calculation path already handles decimals.

Recommendation:

Normalize `collateralAmount` to 18 decimals before passing it to `LibYield.calculateAndRecordYield()` at all call sites:

```
SOLIDITY
// In PoolFacet._harvestLSTYield():
uint256 normalizedAmount = _normalizeToDecimals18(
    lstInfo.collateralAmount, lstInfo.decimals
);
yieldUSD = LibYield.calculateAndRecordYield(
    _lstToken, normalizedAmount, currentRate, bnbPriceUSD
);
```

Apply the same normalization in `StabilityPoolFacet._calculateAndRecordYield()`, `StabilityPoolFacet._previewHarvest()`, and `YieldFacet.previewHarvest()`. Consider adding a shared helper or moving `_normalizeToDecimals18` into a library so all facets can use it without duplicating the function.

Fix Review Notes:

`YieldFacet.previewHarvest()` is the only remaining call site that uses raw `lstInfo.collateralAmount` without `normalizeToDecimals18`. All state-changing yield paths are fixed. The preview function might not cause any issue as it is just a view function, yet it will return incorrect preview values for non-18-decimal LSTs.

E03: Router overview claims PancakeSwap V2 support but only V3 is implemented

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/docs/ROUTER.md>

Description:

The Router overview section states the protocol supports both PancakeSwap V2 and V3 ("支持 PancakeSwap V2/V3"). However, the current codebase exclusively uses the V3 router. There are no `IPancakeV2Router` imports, no V2 swap paths, and no V2 routing logic anywhere in the contract. The v2.0.0 changelog within the same document correctly notes the migration to V3 encoded paths, but the summary section was never updated to reflect this change.

This means the overview contradicts the changelog in the same file. Any integrator reading the summary would expect V2 swap paths to be available, when in reality only V3 is supported.

Potential Benefit:

Correcting the overview eliminates confusion for integrators who may attempt to route swaps through PancakeSwap V2 pools or expect V2-compatible interfaces. It also brings the summary in line with the changelog that already exists in the same document, ensuring the spec is internally consistent.

Recommendation:

Update the Router overview section to reflect V3-only support:

DIFF

```
- 支持 PancakeSwap V2/V3  
+ 支持 PancakeSwap V3
```

Optionally, add a brief note that V2 support was removed in v2.0.0 to give context for anyone familiar with earlier versions of the protocol.

Fix Review Notes:

Not addressed yet.

E04: Public deposit(uint256) described in spec but not implemented in WrappedClisBNB

Location: https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/docs/CLISBNB_INTEGRATION.md#L362

> _CLISBNB_INTEGRATION.md

MARKDOWN

```
360:
361: ```
362: 1. 用户调用 wclisBNB.deposit(1000e18)
363:   ↓
364: 2. wclisBNB 将 1000 slisBNB 从用户转入自己
```

Description:

The integration spec describes a public `deposit(uint256)` function on `WrappedClisBNB` that any user can call directly to transfer `slisBNB`, trigger `provider.provider()`, and receive minted `wclisBNB` in return. Section 5.1 step 1 illustrates this as the primary user flow: "用户调用 `wclisBNB.deposit(1000e18)`".

However, the contract only implements `depositWithCli(address _to, uint256 _amount)`, which is restricted to `ROUTER_ROLE`. No public `deposit()` function exists. The `IWrappedClisBNB` interface declares `deposit(uint256)` but the contract never implements it, meaning the interface and the implementation are out of sync.

The entire "直接 deposit" user path described in the spec is impossible. Users must route all `wclisBNB` deposits through the Router contract. Any integrator or frontend developer building against the documented flow would encounter a revert when calling `deposit()` directly on the `WrappedClisBNB` contract.

Potential Benefit:

Correcting this prevents external integrators and frontend developers from building against a non-existent entry point. It also resolves the mismatch between `IWrappedClisBNB` and the actual contract, which could confuse auditors or tooling that validates interface compliance. Accurate documentation of the Router-only deposit path ensures third-party integrations work on the first attempt.

Recommendation:

Update sections 4.2 to 4.3 and 5.1 in the integration spec to reflect that deposits go through the Router, not directly on `WrappedClisBNB`:

DIFF

```
- 用户调用 wclisBNB.deposit(1000e18) → transfers slisBNB → calls provider.provider() → mints wclisBNB
+ 用户调用 Router.depositClisBNB(amount) → Router calls wclisBNB.depositWithCli() → mints wclisBNB
```

Remove `deposit(uint256)` from the `IWrappedClisBNB` interface, or implement it if a public deposit path is intended. The interface should match the contract.

Alternatively, if the public `deposit()` path was intentionally removed, add a note explaining that all deposits are Router-gated and why (e.g., access control, accounting consistency).

Fix Review Notes:

Not addressed yet.

E05: Documentation mismatch, `minHarvestInterval` default is 1 hour in code, not 0 as documented

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/docs/DEPLOYMENT.md>

Description:

The deployment guide documents `minHarvestInterval` with a default value of `0`, described as having no restriction on harvest frequency. However, `LibYield.initYieldConfig()` sets `ys.minHarvestInterval = 1 hours`, meaning yield harvesting is rate-limited to once per hour by default. This contradicts the documented behavior. A freshly deployed protocol will reject rapid sequential harvests, which operators following the deployment guide would not expect.

The parameter is admin-configurable via `setMinHarvestInterval()`, so this is not a functional defect. However, anyone deploying or reviewing the protocol based on the documentation alone would have incorrect expectations about the initial harvest behavior.

Potential Benefit:

Correcting this ensures operators understand that harvesting is rate-limited from deployment, avoiding unexpected reverts during initial testing or integration. It also prevents auditors from flagging a false discrepancy between the spec and the implementation.

Recommendation:

Update the initialization parameters table in `docs/DEPLOYMENT.md` to reflect the actual default:

```
- | minHarvestInterval | 0 (默认无限制) | ...  
+ | minHarvestInterval | 1 hours (3600 秒) | 收益收割最小间隔 |
```

DIFF

Fix Review Notes:

Not addressed.

E06: MEV protection uses block-based API but docs reference non-existent time-based API

Location: <https://github.com/aspan-finance/aspan-sc/blob/edbbecc9583b48dbfc56d36d250c1a18f70b23f8/docs/SOP.md#L365>

> _SOP.md

MARKDOWN

```
363: | `setFeeTiers(tiers[])` | 批量重设所有层级 | 大规模费率调整 |
364: | `setMaxPriceAge(seconds)` | 设置价格最大有效期 | Oracle 配置调整 |
365: | `setMinDepositPeriod(seconds)` | 设置 MEV 保护期 | 防三明治攻击调整 |
366: | `pause()` | 暂停协议 | 紧急情况 |
367: | `unpause()` | 恢复协议 | 问题修复后 |
```

Description:

The MEV sandwich-attack protection was refactored from time-based to block-based in the Jan 2024 slippage/MEV upgrade (see [CLAUDE.md](#)), but [SOP.md](#) and [DEPLOYMENT.md](#) were never updated.

The codebase stores the MEV protection threshold as `minDepositBlocks` with a default value of 3, initialized in [DiamondInit.sol:55](#). The value is configured through `ConfigFacet.setMinDepositBlocks(uint256 _minBlocks)` ([ConfigFacet.sol:289](#)) and read via `ConfigFacet.getMinDepositBlocks()` ([ConfigFacet.sol:402](#)). The enforcement logic in [LibAppStorage.sol:147](#) compares `block.number` against `lastDepositBlock + minDepositBlocks` to determine whether a withdrawal is permitted.

In docs [SOP.md:365](#) lists `setMinDepositPeriod(seconds)` as the MEV configuration function, but this function does not exist in the codebase and any call to it would revert. [DEPLOYMENT.md:63](#) describes the initialization parameter as `minDepositPeriod = 5 minutes (300 sec)`, using the wrong parameter name and unit. [DEPLOYMENT.md:356](#) provides a code example calling `ConfigFacet(diamond).setMinDepositPeriod(300)`, which would also revert on execution.

Potential Benefit:

Anyone following the current SOP or deployment docs to configure MEV protection will hit a revert on a non-existent function selector. Fixing this eliminates failed ops transactions, removes confusion for new engineers and auditors, and corrects the documented lockout duration (docs say 5 minutes; actual is approximately 9 seconds on BSC). This also prevents future audits from flagging a false spec-vs-implementation discrepancy.

Recommendation:

Apply these three changes:

In [docs/SOP.md:365](#) replace:

```
| setMinDepositPeriod(seconds) | 设置 MEV 保护期 | 防三明治攻击调整 |
```

with:

```
| setMinDepositBlocks(uint256 _minBlocks) | 设置 MEV 保护区块数 | 防三明治攻击调整 |
```

In [docs/DEPLOYMENT.md:63](#) replace:

```
| minDepositPeriod | 5 minutes (300 秒) | MEV 保护 : 存款后锁定期 |
```

with:

```
| minDepositBlocks | 3 blocks (~9s on BSC) | MEV 保护 : 存款后锁定区块数 |
```

In [docs/DEPLOYMENT.md:355-356](#) replace:

```
// 调整 MEV 保护锁定期 (默认 5 分钟)  
ConfigFacet(diamond).setMinDepositPeriod(300);
```

SOLIDITY

with:

```
// 调整 MEV 保护区块数 (默认 3 区块, BSC 上约 9 秒)  
ConfigFacet(diamond).setMinDepositBlocks(3);
```

SOLIDITY

Optionally, add a documentation review step to the upgrade checklist so SOP and DEPLOYMENT docs are verified against the contract ABI on each facet change.

Fix Review Notes:

Not addressed yet.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of EVM smart contracts. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your smart contracts operate securely and as intended.

1. Smart Contract Context and Architecture Analysis

Our auditors begin by deeply examining your EVM smart contract's documentation and intended functionality. We meticulously map out contract interactions, transaction processing flows, and state management logic. Special attention is given to understanding how your smart contract interfaces with critical EVM standards, such as ERC-20, ERC-721, or ERC-4626 tokens, and governance/timelock modules. Last but not least we check external integrations with other DeFi protocols and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for the EVM execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from EVM-specific issues, including but not limited to:

- Unauthorized state variable modification
- Improper Access Control, role checks (`onlyOwner`, `require(hasRole)`) or `msg.sender` verification

- Misuse of low-level calls (`call`, `delegatecall`) or proxy/upgradeability patterns (e.g., storage collisions)
- Incorrect use of external calls, including reentrancy vectors
- Risks stemming from proxy initialization logic or denial-of-service related to gas limitations

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Solidity-based EVM smart contracts. This involves a comprehensive line-by-line inspection of source code, focusing on common EVM vulnerabilities including, but not limited to:

- Missing or insufficient Access Control and Role Checks
- Inadequate `msg.sender` and transaction origin checks
- Incorrect handling of external calls and invocation privileges, particularly potential reentrancy risks
- Arithmetic and integer overflow or underflow errors
- Unsafe use of ABI Encoding/Decoding or low-level assembly
- Improper ERC-20/Token Handling (e.g., approval logic, flash loan vectors, token locking)
- Logic flaws in financial operations or state transitions
- Edge-case handling in function input validation
- Potential denial-of-service vectors related to gas usage and transaction execution

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with EVM best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the EVM development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.